Effective Fuzzing-based Prototype Pollution Detection via Forced Execution and Template Synthesis

DEZHEN KONG, The State Key Laboratory of Blockchain and Data Security, Zhejiang University, China
PEISEN YAO, The State Key Laboratory of Blockchain and Data Security, Zhejiang University, China
JIAKUN LIU, Faculty of Computing, Harbin Institute of Technology, China
LINGFENG BAO*, The State Key Laboratory of Blockchain and Data Security, Zhejiang University, China

Prototype pollution is a critical class of taint-style vulnerabilities in JavaScript programs, enabling attackers to tamper with object prototypes and thereby alter program behavior in unexpected and often dangerous ways. Despite its severity, existing detection techniques struggle with excessive false positives and poor scalability.

In this work, we present Forecast, a lightweight fuzzing-based approach that integrates both forced execution based dynamic analysis and template-based exploit generation to effectively detect prototype pollution vulnerabilities and generate working exploits. Forecast enables better flexibility and scalability and avoids imprecise modeling of JavaScript syntax in static analysis.

Our evaluation of curated benchmarks and real-world Node.js packages shows that Forecast identifies more vulnerabilities than prior approaches, including Explode.js, Graph.js, ObjLupAnsys, and ODGen. Specifically, on the benchmark of known vulnerabilities from prior works, Forecast can identify 228 out of 238 vulnerabilities and generate working exploits for them. Forecast also discovers 24 zero-day vulnerabilities between over 60k popular packages collected from npm registry. Additionally, compared to ODGen and ObjLupAnsys, Forecast demonstrates better scalability by mitigating time-out for many packages.

CCS Concepts: • **Security and privacy** → **Software and application security**; • **Software and its engineering** → *Software libraries and repositories.*

Additional Key Words and Phrases: Prototype Pollution, JavaScript, Forced Execution

ACM Reference Format:
Dezhen Kong, Peisen Yao, Jiakun Liu, and Lingfeng Bao. 2026. Effective Fuzzing-based Prototype Pollution Detection via Forced Execution and Template Synthesis. *J. ACM* 37, 4, Article 111 (June 2026), 26 pages. https://doi.org/XXXXXXX.XXXXXXX

1 INTRODUCTION

JavaScript is a highly dynamic language with complex semantics that pose significant challenges for program analysis [4, 21, 23]. One critical feature that is not present in other popular programming languages is *prototype chain*, i.e., any property lookup does not end up with the present object but goes further up to traverse a chain of the prototypical objects [14, 23]. The prototype chain mechanism can cause a security issue: the prototype of an object (typically `Object.prototype`) can

*Also with Hangzhou High-Tech Zone (Binjiang) Institute of Blockchain and Data Security.

Authors' addresses: Dezhen Kong, The State Key Laboratory of Blockchain and Data Security, Zhejiang University, Hangzhou, China, timkong@zju.edu.cn; Peisen Yao, The State Key Laboratory of Blockchain and Data Security, Zhejiang University, Hangzhou, China, pyaoaa@zju.edu.cn; Jiakun Liu, Faculty of Computing, Harbin Institute of Technology, Harbin, China, jiakunliu@hit.edu.cn; Lingfeng Bao, The State Key Laboratory of Blockchain and Data Security, Zhejiang University, Hangzhou, China, lingfengbao@zju.edu.cn.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM 0004-5411/2026/6-ART111
https://doi.org/XXXXXXX.XXXXXXX

be accessed via special properties like `"__proto__"` and `"constructor"`, which further causes the other properties on the prototype to be redefined. This security issue is referred to as the prototype pollution vulnerability (CWE-1321 [26]), a subtype of taint-style vulnerabilities.

Consider a simple statement `obj[a][b] = c`, supposing that `obj` is created via an object initializer, an adversary sets `a` to `"__proto__"` and `b` to `"toString"`. Then the expression `obj[a]` will refer to the prototype of an ordinary object (i.e., `Object.prototype`). Then the value of the property `"toString"` is re-assigned with the string represented by `c`, e.g., `"polluted"`. This will cause the code `{}.toString()` to trigger an error, since the value of the property `"toString"` is no longer a function and thus cannot be invoked. Such prototype pollution can lead to more severe consequences, e.g., denial-of-service (DoS) attacks and arbitrary code execution. Some well-known JavaScript libraries have been affected by this vulnerability, such as *lodash* 4.17.4 and *merge-deep* 3.0.0.

Several approaches have been proposed to address prototype pollution vulnerabilities, including those based on abstract interpretation and taint analysis [8, 14, 18, 23], symbolic execution [25], and simple rule-based fuzzing. However, the existing approaches are limited by some issues. First, existing approaches typically support only a subset of JavaScript language and framework features, due to the language's inherent complexity [14], leading to both false positives and false negatives. Specifically, ObjLupAnsys [23] and Explode.js [25] do not model a large number of functions in Node.js standard libraries. Second, scalability remains a significant challenge: ObjLupAnsys [23] and Graph.js [8] cannot scale to larger applications, e.g., ObjLupAnsys times out when analyzing the well-known library *lodash* due to significant time consumption in abstracting the whole program, since they try to achieve branch sensitivity. Third, the existing approaches other than ObjLupAnsys are designed for general taint-style vulnerabilities rather than prototype pollution. However, prototype pollution differs from other taint-style vulnerabilities (typically command injection and path traversal). Typically, prototype pollution involves multiple tainted paths, and the sink points are not easily determined. Specifically, the taint-style vulnerability detection tool FAST [14] and NodeMedic [3] are designed *not* to handle prototype pollution vulnerabilities.

As a remedy, we present Forecast[1], a hybrid fuzzing-based framework for detecting prototype pollution with the help of forced execution based dynamic taint analysis and input template synthesis. Forecast can synthesize concrete exploits to trigger the prototype pollution vulnerabilities. Forecast comprises two iterative steps: (i) fuzzing by forcefully executing the program from the detected entry points to collect runtime operations on undetermined input, potential sink information, and (ii) fuzzing by synthesizing grammar-based template strings to refine the function call templates. Initially, the inputs to a public function are all undetermined, and across multiple forced-execution iterations, Forecast can infer function call templates that may trigger the vulnerability. Forecast then refine the function call templates by generating a pool of string templates with placeholders. By filling concrete random strings or dangerous strings like `"__proto__"` into the placeholders, Forecast can synthesize a working exploit.

The above steps offer the following key advantages in detecting prototype pollution compared to the existing approaches:

- *Flexibility*. Forecast offers 3 aspects of flexibility: (i) the structure of the inputs are not limited to fixed static templates, but inferred from the forced execution process, (ii) the structures of strings in the input can be constructed by synthesizing the templates rather than statically defined, (iii) the dynamic nature of forced execution can alleviate the need for a complete static model of JavaScript semantics, making it resilient to language features that are difficult to analyze statically. These can make Forecast applicable to more JavaScript programs and detect more vulnerabilities.

[1]Standing for **for**ced ex**ec**ution and templ**a**te **s**yn**t**hesis.

- *Scalability*. FORECAST employs template synthesis to generate structurally diverse yet semantically focused inputs. Rather than aiming for exhaustive path coverage, it targets execution paths likely to reach sensitive sinks, as identified through taint analysis. This targeted exploration improves the likelihood of triggering vulnerabilities while avoiding the high overhead associated with constraint-solving-based approaches [22, 23, 25].
- *Precision.* FORECAST generates inputs that are both syntactically valid and semantically aligned with vulnerability conditions. This improves detection precision and facilitates exploit generation and manual confirmation.

We conduct extensive experiments on two benchmarks: vulnerabilities from prior work (238 in total) and 5,000 real-world packages collected from the npm repository with high download counts (over 2,000 per week). The evaluation results show that FORECAST can successfully detect and generate working exploits for more prototype pollution vulnerabilities on both benchmarks than other approaches (i.e., GRAPH.JS [8], EXPLODE.JS [25], ODGEN [23], OBJLUPANSYS [22]). FORECAST can generate many more exploits for the vulnerable packages than EXPLODE.JS to validate the existence of prototype pollution vulnerabilities. Specifically, in the first benchmark, FORECAST can detect and generate working exploits for 228 out of 238 prototype pollution vulnerabilities. On the second benchmark consisting of real-world npm packages, FORECAST can detect 24 zero-day vulnerabilities that have not been reported before. Additionally, FORECAST scales better across many JavaScript packages, whereas ODGEN and OBJLUPANSYS time out (e.g., *lodash*, *yargs-parser*, and *fast-json-patch*).

Our contributions can be summarized as follows:

- We propose a new approach FORECAST to detecting prototype pollutions in JavaScript programs and generating exploits that feature forced execution-based analysis and template synthesis.
- We evaluate our approach on both known vulnerabilities and real-world packages, demonstrating its effectiveness in achieving higher true positives, reduced false positives, improved efficiency, and the exploitation of numerous vulnerabilities.

The repository of FORECAST is located at [20].

2 MOTIVATION

In this section, we present motivating examples to illustrate the weaknesses of current approaches and how our approach FORECAST detects the vulnerability and generates a working exploit as well as avoids false positives.

2.1 An Example of Undetected Prototype Pollution Vulnerability

We present the prototype pollution vulnerability[2] in *dot-prop* 2.0.0 as an example. Part of the source code is shown in Figure 1. The function `set` is vulnerable: at the beginning of the execution of `set`, the arguments are set to an object, a string, and any value, respectively. If the second argument `path` is set to a malicious string `"__proto__.polluted"`, the property `"polluted"` of `Object.prototype` will be defined, and the corresponding value is set to `value`.

When implementing `set`, `path` is first split by a dot, yielding an array `pathArr`. The string elements in the array will be processed one by one. If the current element is not the last one, a property lookup is performed on `obj`, and `obj` refers to the corresponding value. However, if the current string element equals a special string `"__proto__"`, `obj` will refer to `Object.prototype`. When the next string element is processed (if it is the last one), a property on `Object.prototype` will be redefined (line 13), thereby triggering prototype pollution. Note that there is not only one pollution path; another

[2]https://nvd.nist.gov/vuln/detail/CVE-2020-8116

```
1  function isObject(x) {
2      return typeof x === 'object' && x !== null;
3  }
4
5  module.exports.set = function (obj, path, value) {
6      if (!isObject(obj) || typeof path !== 'string') {
7          return;
8      }
9
10     var pathArr = path.split('.');
11     pathArr.forEach(function (path, index) {
12         if (!isObject(obj[path])) {
13             obj[path] = {};
14         }
15         if (index === pathArr.length-1) {
16             obj[path] = value; // sink
17         }
18         obj = obj[path];
19     });
20 };
```

Fig. 1. Part of the source code of the package *dot-prop* 2.0.0

malicious string for `path` can also exploit the vulnerability: `"constructor.prototype.polluted"`, since the first property lookup of `"constructor"` on an ordinary object `{}` will yield `Object`.

Challenges. The former state-of-the-art approaches, including ObjLupAnsys [22], ODGen [23] and Graph.js [8], cannot detect the vulnerability in *dot-prop* 2.0.0. And Explode.js [25] cannot generate a working exploit for the vulnerable package. These abstract interpretation-based approaches ***necessitate precise modeling of JavaScript syntax and standard libraries for taint propagation***. However, JavaScript's semantics, such as implicit type coercion, reflective property access, and higher-order functions, are difficult to model and are not well supported by current approaches. In this example, the iteration over `pathArr` is implemented via a higher-order function call to `forEach`, which these tools do not support. Interestingly, if the implementation is changed to `for` or `while` loop, the tools can detect this vulnerability.

The symbolic execution based approach Explode.js still cannot generate a working exploit, since it does not infer a valid function call template for `set` using VIS (vulnerable interaction scheme) [25], i.e., the first argument should be an ordinary object and the second argument should be a symbolic string.

2.2 An Example of False Positive

We present *json-ptr* 1.1.0 as an example; the relevant part of the source code is shown in Figure 2. ObjLupAnsys reports a false positive sink location at line 10 (ODGen times out and Graph.js encounters an error in detecting the prototype pollution vulnerability). The actual sink is in another branch. By analyzing the source code, in the branch from line 5 to 16, `it` refers to an array; `it.length` and `p` are integers. Hence `it[p]` should only refer to an element in the array, rather than the internal prototype objects such as `Array.prototype` and `Object.prototype`. Thus, in the next iteration, `it` cannot refer to those prototype objects, and the statement `it[p] = val` will not cause prototype pollution.

```
function setValueAtPath (target, val, path, force) {
    ...
    while (++cursor < len) {
        step = path[cursor];
        if (Array.isArray(it)) {
            p = toArrayIndexReference(it, step);
            if (it.length > p) {
                if (cursor === end) {
                    rem = it[p];
                    it[p] = val; // false positive sink
                    return rem;
                }
                it = it[p];
            }
            ...
        } else {
            /* actual sink in this branch */
        }
        ...
}
```

Fig. 2. A false positive example of the package *json-ptr* 1.1.0

Challenges. For the static analysis-based approach, like ObjLupAnsys, ***the limitations in language feature support will cause insufficient constraint modeling***. In this example, ObjLupAnsys does not model the type constraints that (i) `it` should be an array, (ii) some variables and expressions, such as `p` and `cursor`, should be a number rather than a string. However, fully modeling the constraints is challenging, requiring consideration of complex and non-trivial JavaScript syntax and internal functions such as `Array.isArray`[3]. Even when a taint flow from the source to sink is detected, synthesizing a working payload that satisfies the constraints in execution paths still needs modeling of JavaScript syntax and string operations.

2.3 Overview of Forecast

To address the challenges, we design Forecast as a hybrid, lightweight fuzzing approach that combines forced execution and grammar-based input synthesis, without requiring static modeling of language features or solving constraints over execution paths. We illustrate here how Forecast detects and generates working exploits, and how it avoids false positives using the examples from prior sections.

2.3.1 How Forecast Detects the Vulnerability. For the vulnerable package *dot-prop* 2.0.0, Forecast begins by identifying the exported function `set` as the entry point since it is directly exported. Then Forecast tries to set different types of values as the arguments of the function `set`.

Stage 1: Forced Execution-based Fuzzing. The overall target in this stage is to infer how the APIs in this JavaScript can be utilized. This stage also makes the execution reach as much code as possible and collects string-related operations for further fuzzing. Forecast leverages a forced execution technique that has the following features:

(1) The object argument is a *proxy object*, i.e., it can be enriched with additional properties as needed.

[3]https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Array/isArray

(2) The string argument is *symbolic*, i.e., the operations involving such a symbolic string can yield other symbolic values.

In a particular execution, FORECAST the arguments `obj`, `path`, and `value` to a proxy object, a symbolic string, and another proxy object, respectively. Since the predicate in line 6 is not satisfied, execution continues.

- In line 10: there is a method call on `path`. Since `path` refers to a symbolic string, this call yields an array containing a random number of symbolic strings, and the split operation will be collected.
- In line 11: there is a method call (`forEach`) on `pathArr`, and the symbolic strings in the array are then processed one by one.
- In line 12: `path` refers to a symbolic string, and it appears in a dynamic property access expression. The expression can yield the value of any property owned by `obj` or `obj`'s prototype (i.e., `Object.prototype`) since we assume that `path` can be any string value set by the user. Hence, FORECAST can assume `path` to be `"__proto__"` and yield `Object.prototype`. Then the two `if`-branches from line 12 to 17 will be skipped since the predicates are not satisfied.
- In line 18: since `path` is now forcefully set to `__proto__`, `obj` refers to `Object.prototype`. When the next element is processed, FORECAST chooses to yield `undefined` for `obj[path]` and `path` is set to a random string, then we can observe that some property of `Object.prototype` is defined.

Such a forced execution successfully reaches the sink location, although it is not feasible in practice. Hence FORECAST can synthesize a function call template (where **ŝ** indicates a symbolic string):

```
require("dot-prop").set({}, ŝ, {});
```

Since there is no return value from the call, FORECAST does not try to discover other methods defined on the returned value to evaluate further.

Stage 2: Format String Generation. In the first stage, the split operation (by `"."`) on the input string `path` is collected. Hence, FORECAST tries to generate a series string split by a dot, e.g., `"{}.{}"`, `"{}.{}.{}"`. Then FORECAST can replace the symbolic string **ŝ** with these format strings, yielding a pool of *ready function call template*. Note that there can be more complicated operations on a symbolic string other than *split*. FORECAST can synthesize more complex template strings (e.g., replace a placeholder with other templates).

Phase 3: Exploitation. The placeholders in a ready function call template should be replaced with a concrete string, i.e., a random string or a dangerous string `"__proto__"`, `"constructor"` or `"prototype"`. Finally, FORECAST generates a working exploit:

```
require("dot-prop").set({}, '__proto__.polluted', {});
```

2.3.2 How FORECAST Avoids False Positives. For the false positive example in Section 2.2, no matter in forced executions or actual executions, in the if-branch from line 5 to 16, `it` is always an array and `p` is always a number. Hence, the prototype object `Object.prototype` can never be accessed via `it[p]` and assigned back to `it`. FORECAST will never report the false positive case.

3 APPROACH

This section presents our fuzzing-based approach, FORECAST, for detecting prototype pollution vulnerabilities. The overall workflow is shown in Figure 3. FORECAST comprises the following stages.

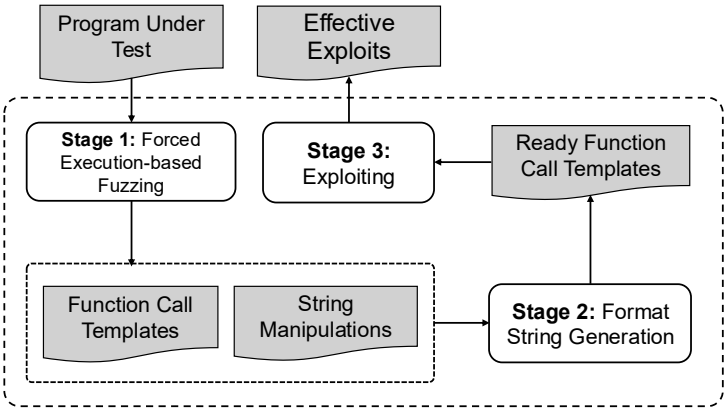


Fig. 3. Workflow of Forecast

- Stage 1: forced execution-based fuzzing to infer function call templates that model the API usage of the provided functions.
- Stage 2: format string generation to refine function call templates.
- Stage 3: exploit generation based on the ready templates.

We emphasize that our approach does not opt for soundness or completeness. Rather than aiming for exhaustive detection, we prioritize practical effectiveness in identifying concrete, exploitable flows in real-world JavaScript programs under our practical threat model.

3.1 Preliminary

In this section, we explain the definition of the concepts in Forecast. We formalize the overall process as a problem of finding possible *function call templates*, and generating concrete exploits based on them. We introduce function call template to model how APIs in a JavaScript package can be utilized to exploit a prototype pollution vulnerability inspired by API access paths in prior work [27, 28, 39].

The API access paths in prior works are used to demonstrate how a public function can be loaded or interact with arguments. Forecast extends the API access paths to conform to typical real-world usage, especially supporting both synchronous and asynchronous function calls and complicated sequential calls in a JavaScript library.

Definition 3.1 (Function Call Template). A function call template is either the imported library (if the entry can be directly called) or a function call template with another function call appended. The function call can be asynchronous or asynchronous (`then(...)` call).

The detailed syntax is shown in Figure 4, where $\bar{\cdot}$ denotes repeated items (can be concatenated by a comma) and $\hat{\mathbf{s}}$ denotes a symbolic string value, which will be used in forced execution. Note that we only allow simple value forms since we focus on a practical threat model for prototype pollution vulnerabilities in Node.js, i.e., the arguments be simple and conform to typical usage, as the following assumptions states below.

Assumption 1. *An exported function should be invoked where each input argument is either a primitive value or an ordinary object constructed by* `Object`.

Assumption 2. *An exploit only includes the functions exported by the library.*

$$
\begin{array}{rcll}
T & ::= & \texttt{require(}\,str\,\texttt{)}\ \overline{c} & \text{template with sync calls} \\
 & | & \texttt{require(}\,str\,\texttt{)}\ \overline{c}\ ac & \text{template with async calls} \\
c & ::= & \texttt{(}\,\overline{v}\,\texttt{)} \mid \texttt{.}\,str\,\texttt{(}\,\overline{v}\,\texttt{)} & \text{sync call pattern} \\
ac & ::= & \texttt{.then(}\,f\,\texttt{)} & \text{async call pattern} \\
f & ::= & \texttt{function(result) \{ result}\,\overline{c}\,\texttt{\}} & \\
 & | & \texttt{function(result) \{ result}\,\overline{c}\ ac\,\texttt{\}} & \\
 & | & \texttt{function() \{\}} & \text{empty function} \\
v & ::= & \texttt{undefined} \mid \texttt{null} \mid \texttt{true} \mid \texttt{false} & \text{primitive values} \\
 & | & str \mid num & \text{string or number} \\
 & | & \texttt{[}\,\overline{v}\,\texttt{]} & \text{array} \\
 & | & \texttt{\{}\,\overline{str\texttt{:}v}\,\texttt{\}} \mid \texttt{\{}\,\hat{\mathbf{s}}\texttt{:}v\,\texttt{\}} & \text{object}
\end{array}
$$

Fig. 4. Syntax of function call template

The callback-style API is similar to `then`-style usage, hence we do not include it in the function call template since it can be simply transformed to async-style usage. Stage 1 of Forecast aims to generate such function call templates. We present several examples.

Example 3.2. If a JavaScript package named *foo* can be directly called after it is imported, with a string and another configurable object as arguments, one can write the following template:

```
require("foo")(ŝ, dot: true);
```

Example 3.3. If a JavaScript package named *bar* provides a function named `setProp`, which can be called with a string. The returned value has a function property named `setValue` and can be called with the returned value as *this* object. One can write the following template:

```
require("bar").setProp(ŝ).setValue(ŝ).test();
```

Example 3.4. If a JavaScript package named *bar* provides a function named `merge`, which can be used in Promise-style (the resulting object can be obtained by `then` calls). The function accepts an object with no properties as the first argument, and another object that can have a randomly-named property (currently represented by symbolic string $\hat{\mathbf{s}}$) as the second argument.

```
require("bar").merge({}, {ŝ: {ŝ: ŝ}).then(function(result) { ... });
```

The symbolic string should be replaced with concrete format string. Thereby Forecast can further construct concrete exploit according to the function call template. We require the malicious strings follow typical cases, i.e., directly including the property names to be accessed.

Assumption 3. *The attacker-controlled string contains the malicious tokens directly, without requiring further computation or encoding. In other words, the tainted strings used in property access expressions can be directly extracted from the input source.*

Example 3.5. In Section 2, the malicious payload `path` straightforwardly contains the sensitive strings that can pollute the prototype (i.e., `"__proto__"` and `"polluted"`).

Definition 3.6 (Ready Function Call Template). A function call template is *ready* if the symbolic values $\hat{\mathbf{s}}$ are replaced with a concrete string that may contain several placeholders. A placeholder can be unlimited (represented by `"{}"`) or a regular expression.

Algorithm 1: Process of Forced Execution based Fuzzing

Input: A JavaScript library *lib* and loaded object *loadedPackageObj*
Output: Accepted function call templates

```
1  procedure ForcedExecutionFuzz()
2      tmplFuncs ← an empty set
3      if loadedPacakgeObj is a function then
4          tmplFuncs.add({ require(lib), loadedPackageObj })
5      else
6          funcNames ← GetFuncProps(loadedPackageObj)
7          for n in funcNames do
8              tmplFuncs.add({ tmpl.concat(n), null })
9      acceptedTmpls ← an empty array
10     while tmplFuncs is not empty do
11         { tmpl, obj } ← tmplFuncs.pop()
12         while execution time does not exceed the limit do
13             { polluted, resultObj, input } ← ForcedExecution(tmpl, obj)
14             if polluted == true then
15                 acceptedTmpls.add(tmpl.concat(input))
16             else
17                 funcNames ← GetFuncProps(resultObj)
18                 for n in funcNames do
19                     tmplFuncs.add({ Concat(tmpl,input, n), resultObj })
20     return acceptedTmpls
```

Example 3.7. Forecast can generate a ready template for the function call template in Example 3.4:

```
require("bar").merge({}, {"{}": {"{}": "{}"}).then(function(result) { ... });
```

These placeholders should be replaced with a random string or some dangerous string, i.e, `"__proto__"`, `"constructor"`, or `"prototype"` if it is a free placeholder `"{}"`. If the placeholder is a regular expression, it can only be replaced with a string that follows the regular expression.

3.2 Stage 1: Forced Execution-based Fuzzing

In this stage, Forecast leverages forced execution for fuzzing, to help find suitable function call templates. This technique enables lightweight taint propagation to identify the potential sinks and determine potential function calls that can lead to prototype pollution, even in the absence of precise abstract semantics for JavaScript. Designing such semantics is inherently challenging due to JavaScript's highly dynamic nature, which complicates traditional static analysis.

3.2.1 Overall Procedure. Algorithm 1 exhibits the procedure of forced execution-based fuzzing, comprising two parts, i.e., initialization and major loop.

Lines 2 to 8 outline the initialization of function call templates. The list *tmplFuncs* is to store the potential functions that can be used to construct a function call template (e.g., `require("foo").setProp` in Example 3.3), and *acceptedTmpls* is to store the accepted templates. If the loaded library is a function, it is added to *tmplFuncs* directly (line 4); otherwise, the functions inside the object will be added.

Lines 10 to 19 illustrates the main loop: each iteration retrieves *tmpl* and *obj* from *tmplFuncs* list, which stands for a candidate function and the corresponding *this* (the prior returned object). Then FORECAST forcefully executes the function (line 13, cf. Section 3.2.2). If this execution causes one of the common prototypes polluted, the template will be added to accepted template list (line 15); otherwise, if the result from the forced execution is not a primitive value or an object constructed by `Object`, FORECAST will expand the current template by searching the function values inside the resulting object and adding them to the candidate function list (line 16 to 19, cf. Section 3.2.3).

Finally, the procedure returns a list of accepted function call templates (line 20) for the next stage (cf. Section 3.3).

3.2.2 Forced Execution Procedure. We here detail the forced execution procedure (i.e., FORCEDEXECUTION in Algorithm 1). In prior work, forced execution features replace unknown or empty values (`null` or `undefined`) with a special proxy value to allow the program to execute as far as possible without crashing. FORECAST extends forced execution by allowing forcefully setting the tainted string to dangerous values to reach the sinks (i.e., pollute the internal objects) and extending the structure of input objects by recording property access and writes. Additionally, FORECAST collects operations on tainted values and objects and producing new tainted values.

Specifically, when the execution of a function f starts, FORECAST will set *this* to `undefined` or a resulting object from another forced execution (discussed in Section 3.2.3), and the arguments to *proxy objects*, a symbolic string value $\hat{\mathbf{s}}$ or other primitive values like a number, a boolean value, etc. The proxy objects are initially empty, and the properties can be added to by need, and $\hat{\mathbf{s}}$ is a special value that represents any tainted string. During a forced execution, FORECAST processes the following critical program structures that may involve proxy objects.

Processing Dynamic Property Access For a property access expression `o[p]=v`, if `o` and `p` both refer to actual values, the expression is executed normally. Otherwise, the overall procedure of processing property accesses is presented in Algorithm 2, where $[\![e]\!]_{\mathcal{S}}$ means the evaluation results of the expression e in the current execution store $\mathcal{S}$. For general cases, if p exists on o, then the corresponding value of the key p is returned directly; otherwise, if o is a proxy object, there is a new property named $[\![p]\!]$ added to o, and the value is randomly generated, which can be a number, a boolean value, $\hat{\mathbf{s}}$, or a new proxy object. In case that the property p is just $\hat{\mathbf{s}}$: if there is already some default property on $[\![e]\!]_{\mathcal{S}}$, the corresponding default value is directly returned. Otherwise, FORECAST randomly chooses a property name on $[\![e]\!]_{\mathcal{S}}$ or $[\![e]\!]_{\mathcal{S}}$'s prototype[4] and returns the corresponding value. This could cause execution to reach the sink and write some property to the internal prototype object.

Processing Dynamic Property Write The overall procedure of property write is exhibited in Algorithm 3. For a property write expression $e[p] = v$, if $[\![e]\!]_{\mathcal{S}}$ is not an object, the operation is discarded. If p does not equal to $\hat{\mathbf{s}}$, the property and the value are written $[\![e]\!]_{\mathcal{S}}$, no matter whether it is a proxy object. Otherwise, a special property named $\hat{\mathbf{s}}$ will be set on $[\![e]\!]_{\mathcal{S}}$. If $[\![e]\!]_{\mathcal{S}}$ is by chance an internal prototype object, FORECAST will report a possible sink, i.e., the forced execution can potentially pollute the prototype object.

Processing Static Property Access and Write The process of static property access and write is similar to that of dynamic property access and write. For non-proxy objects, the execution is ordinary. For proxy objects, the properties are added as needed (like the procedure described by lines 6 to 9 in Algorithm 2). Differently, if in the expression $e.n$ where e evaluates to $\hat{\mathbf{s}}$ and n is a function name in `String.prototype`, FORECAST returns a wrapped function for $e.n$ with

[4]Including `"__proto__"` and `"constructor"`.

Algorithm 2: Processing Property Access in Forced Execution

Input: the expression in the program $e[p]$; execution store $\mathcal{S}$
Output: the value of the property access expression

```
1  procedure PropertyAccess()
2      [[e]]_S, [[p]]_S = Eval(e, S), Eval(p, S)
3      if [[p]]_S is not ŝ then
4          if [[p]]_S exists on [[e]]_S then
5              return GetProp([[e]]_S, [[p]]_S)
6          else if isProxy([[e]]_S) then
7              newValue ← GetRandomValue()
8              SetProp([[e]]_S, [[p]]_S, newValue)
9              return newValue
10         else
11             return undefined
12     else
13         x ← RandomChoose(PropsInChain([[e]]_S).concat(ŝ))
14         if x is not ŝ then
15             return GetProp([[e]]_S, x)
16         else if default property does not exist on [[e]]_S then
17             newValue ← GetRandomValue()
18             AddDefaultProp([[e]]_S, newValue)
19             return newValue
20         else
21             return GetDefaultProp([[e]]_S)
```

Algorithm 3: Processing Property Write in Forced Execution

Input: a property write expression $e[p] = v$; execution store $\mathcal{S}$

```
1  procedure PropertyWrite()
2      [[e]]_S, [[p]]_S, [[v]]_S = Eval(e, S), Eval(p, S), Eval(v, S)
3      if [[e]]_S is not an object then
4          Error()
5      if [[p]]_S is not ŝ then
6          SetProp([[e]]_S, [[p]]_S, [[v]]_S)
7      else
8          AddDefaultProp([[e]]_S, [[v]]_S)
9          if [[e]]_S is an internal prototype object then
10             ReportSink(e[p] = v)
```

this bound to e. When the function is executed, it will collect the function name n and the arguments (if they are concrete values), and return $\hat{\mathbf{s}}$ if `String.prototype.`n returns a string.

Processing Type Coercion Type coercion is the automatic or implicit conversion of values from one data type to another, e.g., when a non-number variable `a` is in a binary operation `a < b`, `a` will be implicitly coerced to a number by calling `a[Symbol.toPrimitive]()`[5]. If there is

[5]https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Symbol/toPrimitive

such a type coercion operation on proxy object, then the execution is interrupted since the case should not be allowed for an ordinary object.

Processing Method Call For a method call $a.f()$ where a does not refer to a proxy object, it is executed as usual. When a refers to a proxy object o, since the function call template in FORECAST only supports one callback argument as the last one, the execution is simply interrupted.

Processing Native Function Calls FORECAST mainly deals with several critical native functions for proxy objects. Regarding the function call `Object.keys(`e`)` [30] where e evaluates to a proxy object o that has no own properties, FORECAST creates a special and randomly generated property p in o. The corresponding value is set to a new proxy object. Then the property list of o will be returned. Otherwise, the current property names will be returned. Other property-related functions, such as `Object.getOwnPropertyNames`, are processed similarly. For string manipulation functions like `encodeURI` [29], FORECAST returns $\hat{\mathbf{s}}$. For regular expression matching functions, including `RegExp.prototype.test` and `RegExp.prototype.exec` when the argument is $\hat{\mathbf{s}}$, FORECAST collects the function names and the corresponding regular expression.

Processing Specific Syntactic Structures FORECAST only extraordinarily considers the following structures. Regarding "`for .. in`" structure, if the proxy object currently has no properties, the iteration yields $\hat{\mathbf{s}}$ and otherwise, yields the property names one by one. For other `for` and `while` loops, FORECAST limits the iteration time to a fixed value, which is also adopted by approximate interpretation [21]. For arithmetic expressions, FORECAST only considers equality "==" and strict equality "===" if one operand can be evaluated to $\hat{\mathbf{s}}$, and another operand will be collected.

In a forced execution, FORECAST also hijacks the runtime environment, especially I/O-related functions, to detect a vulnerability in which the exploit's payload comes from the local file system or standard input. FORECAST mocks the standard library `fs`[6] and `process`[7]. The mocked module `fs` works like the memory-based file system utility *memfs*[8]. Specifically, for `fs.readFile` and `fs.readFileSync`, the resulting buffers or strings are marked as tainted. The files on the virtual file system are created as needed (if there is a `fs.existSync` or `fs.stat` call). The hints come from the read hints and write hints of the original approximate interpretation. The structure maps expressions to a set of locations where they can be evaluated. For standard input and output, FORECAST hijacks `process.stdin` and `process.stdout` by implementing a customized `fs.ReadStream` and `fs.WriteStream`. For other standard libraries, FORECAST processes them as Laursen et al. [21], i.e., simply using a special object to represent them, and for any calls of the function, return that special object[9].

3.2.3 Template Expansion. The process corresponds to lines 16 to 19 in Figure 1. The structure of the input arguments can be restored. If the forced execution cannot lead to potential prototype pollution, however, if there is some property name or value written into the resulting object, the method on the resulting object can be called subsequently, which may trigger the prototype pollution. Hence FORECAST expands the original function call template to append another function call on the resulting object. Continuing Example 3.3: the method call `setProp` will put $\hat{\mathbf{s}}$ into the resulting object, and then the method call `setValue` will also put another $\hat{\mathbf{s}}$ into the resulting object.

[6]See https://nodejs.org/api/fs.html.
[7]See https://nodejs.org/api/process.html.
[8]https://www.npmjs.com/package/memfs
[9]In their original work, this special object is represented by $\mathbf{p}^{\star}$.

Then, the resulting object is used as *this* object for `test` call, where the symbolic string $\hat{\mathbf{s}}$ may propagate to some expression in a property access or write expression.

3.3 Stage 2: Format String Generation

After inferring the function call templates that can possibly lead to prototype pollution, FORECAST attempts to replace the symbolic string values with grammar-based template strings. Our basic observation is that many real-world prototype pollution vulnerabilities follow syntactic patterns in how input strings are processed. By constructing suitable patterns for a symbolic string, dangerous strings can be matched to exploit the vulnerability.

FORECAST extends the symbolic string $\hat{\mathbf{s}}$ in function call templates to concrete strings based on runtime information obtained in the first stage, i.e., the function names on `String.prototype` and the arguments. FORECAST then generates a pool of concrete strings for each symbolic string. For the specific method calls, FORECAST creates a distinct template:

- if the function name is `split` or `indexOf`, create a separator-split string, e.g., if the separator is `"&"`, create the strings like `"{}"`, `"{}&{}"`, `"{}&{}&{}"`, etc.
- if the function name is `match`, FORECAST generates a random string that satisfies the concrete characters in a regular expression, and the other parts (including the matching parts and some structures with asterisk or plus sign, e.g., `[a-z]+`) with a placeholder with the caught part of the regular expression.
- for strict equality operations on $\hat{\mathbf{s}}$ and another concrete token, FORECAST can create a template containing that token, e.g., if the tainted string is compared with `"["`, then FORECAST can generate template strings `"[{}"`, `"{}["` and `"{}[{}"`.

The placeholders in a template can be substituted with other template strings to construct more templates. The consecutive placeholders should be merged. FORECAST also considers JSON, `INI`, XML and YAML-based meta-format strings, e.g., `"\{\"{}\":\"{}\"\}"` and `"<{}>{}</{}>"` to replace with placeholders.

Example 3.8. Consider the package *yargs-parser* that parses the command-line arguments. We present part of the code in Figure 5. There is a regular expression match operation of `"^--.+"` with no catching group (line 12), a regular expression match operation of `"^--?(.+)"` (line 13), a split call (line 7), and a comparison operation with a white space (in function `tokenizeArgString`, which is not presented due to space limitation) on symbolic strings. Hence FORECAST creates the format strings `"--{}"` and `"{}.{}"`, `"{} {}"`, etc. FORECAST can also merge the template strings, creating `"--{}.{}"`, `"--{}.{} {}"`, etc.

Example 3.9. Consider the function `parse` in the package *iniparser* 1.0.5. We present part of the code in Figure 6. There are two regular expression matching calls in lines 10 and 13. For the first matching calls for the regular expression `"^\s*;.*$"`, FORECAST can generate a template string `"{\s*};{}"`. For another matching call for `"^\s*([\w\.\-\_]+)\s*=\s*(.*?)\s*$"`, FORECAST can generate a template string `"{\s*}{}{\s*}={\s*}{}{\s*}"`.

FORECAST generates a set of format strings for a function call template to substitute the placeholders. To ensure performance, a ready-to-use function-call template should have at most 5 placeholders, which is often enough to reach a prototype and pollute it, as the exploits of most vulnerable packages in previous works [1, 22, 23] only needs 3 concrete strings and a small part of packages needs 4 strings.

```
function parse(args, opts) {
  args = tokenizeArgString(args);
  var argv = { _: [] };

  function setArg(key, val) {
    var value = processValue(key, val);
    var splitKey = key.split('.'); // split operation
    setKey(argv, splitKey, value);
  }
  ...
  for (var i = 0; i < args.length; i++) {
    if (arg.match(/^--.+/)) { //regex matching
      key = arg.match(/^--?(.+)/)[1]; // regex matching
      next = args[i + 1];
      if (!next.match(/^-/)) {
        setArg(key, next);
        i++;
      }
      else { /* omitted */ }
    }
    else { /* omitted */ }
  }
```

Fig. 5. Simplified code of *yargs-parser* 6.0.0 (There are 3 string-related operations on the symbolic string)

```
var regex = {
    section: /^\s*\[\s*([^\]]*)\s*\]\s*$/,
    param: /^\s*([\w\.\-\_]+)\s*=\s*(.*?)\s*$/,
    comment: /^\s*;.*$/
};
...
function parse(data) {
    ...
    lines.forEach(function(line) {
        if (regex.comment.test(line)) {
            return;
        } else if (regex.param.test(line)) {
            var match = line.match(regex.param);
            ...
```

Fig. 6. Part of the code in *iniparser* 1.0.5

3.4 Stage 3: Exploitation

After FORECAST generates a series of refined function call templates, it can replace the placeholders with a concrete string. Specifically, if the placeholder is `"{}"`, replace it with a random string or a dangerous string `"__proto__"`, `"constructor"`, or `"prototype"`, otherwise, the placeholder contains a regular expression, FORECAST replaces it with a random string that satisfies the regular expression or a dangerous string that also satisfies the regular expression. In Example 3.9, the placeholder `"{\s*}"` can only be replaced with a string comprising white spaces.

If during the execution, a property of one of the internal objects is redefined, Forecast will report a prototype pollution vulnerability along with the sink location. If the execution fails, Forecast simply discards it.

4 EVALUATION

We evaluate the effectiveness and performance of Forecast in this section. Specifically, we focus on the following research questions.

- **RQ1:** How effective is Forecast in detecting prototype pollution vulnerabilities and generating exploits for existing benchmarks?
- **RQ2:** Can Forecast detect zero-day vulnerabilities and generate working exploits for them?
- **RQ3:** Can Forecast more efficiently detect prototype pollution vulnerabilities and generating exploits?

4.1 Evaluation Setup

4.1.1 Benchmark Construction. To evaluate the effectiveness and performance of Forecast, we construct two benchmarks: the first benchmark comprises the vulnerable packages identified in prior work, and the second comprises popular packages from the npm registry.

Benchmark 1. The vulnerabilities in this benchmark are collected from the following sources:

- The vulnerable packages provided by Ferreira et al. [8, 25], which were extracted from SecBench.js [1] and VulcaN [2]. The original benchmark consists of 188 JavaScript packages, yielding 214 vulnerabilities in total (one sink location corresponds to a vulnerability, and one package can have multiple sinks).
- The vulnerabilities reported by the authors of ObjLupAnsys [22], ODGen [23], Graph.js and Explode.js, consisting of 61, 19, 16, and 5 packages, respectively.
- Legacy vulnerabilities that are used by the evaluation of ObjLupAnsys [22] (52 in total).

We obtain the vulnerabilities mentioned above, and then remove duplicate vulnerabilities with the exact sink location and attack path. For example, the vulnerable package *dotty* 0.0.1 is reported by both Ferreira et al. [8] and Li et al. [22], hence we remove one of them. We also check the correctness of the annotated sinks of all vulnerabilities. Suppose the provided exploit can actually reach the sink location, or we can manually create an exploit to get the sink location and pollute the internal objects. In that case, the annotated sink is identified as "correct". Finally, we retain 238 vulnerabilities in total, involving 206 *npm* package-version pairs (multiple versions of an *npm* package may appear in Benchmark 1, e.g., *assign-deep* 0.4.6 and 0.4.8).

Benchmark 2. For this benchmark, we collect the packages from the official *npm* repository, with over 2,000 weekly downloads, following the guideline used by Ferreira et al. [8]. We also remove the packages that contain no JavaScript source code (e.g., TypeScript declaration packages with only `.d.ts` files). We retain a total of 63,285 packages and fetch the latest version of each package. Note that some packages in Benchmark 2 can also be included in Benchmark 1; however, the corresponding versions are the latest. For example, Benchmark 1 includes *assign-deep* 1.0.0, but Benchmark 2 include version 1.0.1, and this version fixes the prototype pollution in 1.0.0.

4.1.2 Evaluation Settings. We detail the evaluation settings for evaluating Forecast and other state-of-the-art approaches here. For Forecast, we implement the prototype tool in pure JavaScript without native code. Additionally, the maximum number of forced executions for a function and a synthesized function call template is set to 1,000. The max memory size of the V8 engine's old memory section is set to 16 GB. For ODGen and ObjLupAnsys, we require them to analyze the entire npm packages, i.e., the analysis starts from the entry file of each package. For Graph.js and

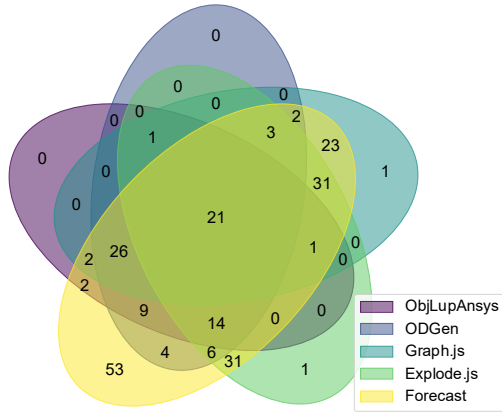


Fig. 7. Venn diagram of the overlap of detected vulnerabilities on Benchmark 1

Explode.js, since they only support a single JavaScript source file per run, we let them analyze the bundled JavaScript files (if not included in the downloaded package, we manually create them using Webpack [40]). The timeout for the above approaches is set to 5 minutes, which is also adopted by Ferreira et al. [8] when they evaluated Graph.js and Explode.js.

4.1.3 Metrics. We consider the true positive (*TP*), false positive (*FP*), and false negative (*FN*) rates for vulnerability detection and the exploitation rate (*E*) for exploit generation. A metric may not fit all approaches. Specifically:

- True positive (*TP*) refers to the number of vulnerabilities that an approach can identify (or the execution can reach) a sink location where one property of an internal prototype can be redefined to a tainted value. Forecast and other evaluated approaches can directly output the sinks.
- False positive (*FP*) refers to the number of incorrectly identified sink points by an approach. Forecast does not produce FPs since it avoids presenting unreachable sink locations.
- False negative (*FN*) refers to the number of vulnerabilities where an approach cannot find an attack path that leads to the sink point.
- Exploit (*E*) means that the rate of vulnerabilities for which an approach can successfully generate an exploit to reach the corresponding sink and pollute the internal objects.

4.2 RQ1: Effectiveness of Vulnerability Detection

Table 1 presents the detection results of Forecast and other state-of-the-art approaches on Benchmark 1, where a dash "-" indicates that the metric is not applicable for the corresponding approach.

Obviously, Forecast detects more prototype pollution vulnerabilities than prior approaches, i.e., 228 TPs in total, whereas Graph.js, ODGen, and ObjLupAnsys only detect 111, 86, and 76 vulnerabilities, respectively. As for Explode.js, which is based on the Multiversion Dependency Graph generated by Graph.js, it can only generate working exploits for 109 vulnerabilities. Figure 7 illustrates the intersection of vulnerabilities detected by the evaluated approaches. Forecast fails to detect only 3 vulnerabilities identified by other tools, while it uniquely discovers 53 vulnerabilities that no other approach detects. Additionally, among those vulnerabilities that no other tool could detect within the 5-minute timeout, Forecast successfully identifies and generates exploits for all

Table 1. Evaluation results on Benchmark 1

Approach	TP	FP	FN	E
Graph.js	111	60	127	-
Explode.js	-	-	-	109
ODGen	86	19	152	-
ObjLupAnsys	76	21	162	-
Forecast	228	-	10	228

of them. These results highlight Forecast's key strength: its ability to reliably produce working exploits for *npm* packages within practical time bounds.

Forecast does not generate working exploits for 10 vulnerabilities on Benchmark 1, which is much lower than other approaches (all of them fail to detect over 100 vulnerabilities). The undetected vulnerabilities can be mainly categorized into the following cases.

- Case 1: Forecast cannot synthesize any effective function call template. For *angular* 1.7.8 and *jquery* 1.11.0, although they are published to *npm*, they require a browser environment (e.g., the `window` global object) to run normally.
- Case 2: There are many exported functions and therefore Forecast cannot check all of them within the maximum time limit. In our evaluation, *mout* and *phpjs* exports too many functions in `mout.object`, `mout.function`, etc., supposing that the loaded library is referenced by `mout`.
- Case 3: Forecast fails to synthesize correct function call template. For example, *@hapi/subtext* needs complicated input (including HTTP request object) and Forecast cannot successfully synthesize the required structure to make the program correctly run.

Graph.js, ODGen, and ObjLupAnsys report 60, 19, and 21 FPs, respectively. The FP cases result from the insufficient modeling of JavaScript features, such as the example presented in Section 2.2. By contrast, Forecast does not produce false positives, since only the actually reached sink locations will be output to the user.

Takeaway: Forecast successfully generates working exploits for 228 out of 238 packages (95.8%) in Benchmark 1, outperforming other approaches.

4.3 RQ2: Effectiveness of Zero-day Vulnerability Detection in Real-world Packages

Table 2 exhibits the evaluation results of Forecast and other approaches on Benchmark 2, in terms of TP, FP and working exploits. To reduce time consumption, we only evaluate other approaches on the vulnerable packages detected by Forecast. Totally, Forecast generates effective exploits for 154 packages on Benchmark 2, where 24 of them are identified as zero-day vulnerabilities (details of these zero-day vulnerabilities are listed in Table 3, where the dash means that the corresponding packages *dot-path-value* and *ember-cli-lodash-subset* are compressed into one line). Note that we regard a vulnerability as *zero-day* if it cannot be found in CVE databases [6], GitHub advisories [9] or Snyk [37].

By contrast, between the 154 vulnerable packages, other approaches can discover no more than 20 vulnerable packages, and only few zero-days, which are the subsets of those discovered by Forecast. Specifically, ObjLupAnsys and ODGen can detect vulnerabilities for only 3 packages, namely *mongo-object* 3.0.1, *framework-utils* 1.1.0 and *jss-plugin-extend* 10.10.0, whereas they time out analyzing other packages. Graph.js can detect 3 vulnerabilities, i.e., those in *mongo-object* 3.0.1, *mongoose-query-parser* 1.3.1, *group-args* 0.1.0 but times out analyzing 8 packages and fails to

Table 2. Evaluation results on Benchmark 2

Approach	TP	Zero-day	FP	E
Graph.js	12	3	4	-
Explode.js	14	2	-	14
ODGen	16	3	7	-
ObjLupAnsys	17	3	6	-
Forecast	154	24	-	154

Table 3. Detected zero-day vulnerabilities (a dash means that the analyzed file is compressed into one line)

Package	LoC	Weekly Download	Sink Location
dot-path-value 0.0.11	-	4,513	-
object-resolve-path 1.1.1	319	15,975	path.js (line 248)
path-object 2.3.0	114	11,042	index.js (line 54)
mongo-object 3.0.1	981	20,698	lib/util.js (line 226)
ember-cli-lodash-subset 2.0.1	-	119,507	-
object-mapper 6.2.0	406	34,739	src/object-mapper.js (line 264)
mongoose-query-parser 1.3.1	398	2,239	lib/index.js (line 166)
prompt 1.3.0	717	716,035	lib/prompt.js (line 348)
@js.properties/properties 0.5.4	4,822	32,028	parse-to-properties.js (line 83)
whet.extend 0.9.9	104	500,368	lib/whet.extend.js (line 32)
utila 0.5.0	872	11,028,682	lib/object.js (line 101)
keypather 3.1.0	2,591	24,555	lib/operations/set.js (line 16)
underscore-plus 1.7.0	522	58,759	lib/underscore-plus.js (line 149)
immutable-ops 0.7.0	2,659	5,994	lib/index.js (line 109)
group-args 0.1.0	98	24,850	index.js (line 45)
factory-girl 5.0.4	17,135	53,739	index.js (line 49)
jss-plugin-extend 10.10.0	169	321,004	dist/jss-plugin-extend.js (line 117)
normalizr 3.6.2	599	638,881	dist/normalizr.js (line 163)
pastable 2.2.1	595	222,769	dist/misc-a5381864.cjs.dev.js (line 88)
framework-utils 1.1.0	93	428,352	dist/utils.cjs.js (line 38)
setvalue 2.1.0	50	7,073	dist/index.js (line 13)
modifyjs 0.3.1	554	9,380	dist/bundle.js (line 360)
typhonjs-object-util 0.4.2	885	3,348	dist/ObjectUtil.js (line 322)
cerialize 0.1.18	741	6,739	dist/serialize.js (line 411)

detect vulnerabilities in 11 packages. Besides, Explode.js can only generate 2 working exploits for *mongo-object* 3.0.1 and *whet.extend* 0.9.9, respectively.

False Positives. Graph.js, ODGen and ObjLupAnsys report 4, 7 and 6 FPs, respectively. The cause of such FPs lies in incorrect modeling of constraints, as illustrated by the example in Section 2.2. Specifically, for ObjLupAnsys, 5 FPs are due to incorrect modeling of key filtering (the latest version of the packages have fixed the prototype pollution by disallowing some dangerous keys) and 1 FP is from incorrect type modeling.

Table 4. The number of finished packages and unfinished packages of each approach (on Benchmark 1)

Approach	Finish (%)	Timed out	Error
Graph.js	201 (84.4%)	28	9
Explode.js	200 (84.0%)	34	4
ObjLupAnsys	144 (60.5%)	93	1
ODGen	160 (67.3%)	76	2
Forecast	235 (98.7%)	1	2

Ethical Consideration. We responsibly report the detected zero-day vulnerabilities to the developers of the corresponding packages and also submit them to Snyk [37], with the descriptions and proof-of-concept exploits to help the review team confirm the existence of vulnerabilities. If we do not receive responses in one month, we also submit them to Mitre[10]. By the time of submission, we have been assigned 4 CVE IDs, i.e., CVE-2025-65462, CVE-2025-65464, CVE-2025-67180 and CVE-2026-16266 for *dot-path-value*, *object-resolve-path*, *ember-cli-lodash-subset* and *mongo-object*, respectively, and 2 vulnerablities (those in *typhonjs-object-util* and *dot-path-value*) have been fixed.

Takeaway: Forecast finds 154 vulnerabilities (with 24 zero-days) between the real-world packages, outperforming other approaches by a large margin.

4.4 RQ3: Efficiency Comparison

Table 4 presents the number of packages where each approach finishes analysis (or generates working exploits for Explode.js and Forecast), times out, or encounters errors during the analysis. Forecast can finish analyses for 98.7% of the packages, while Graph.js and Explode.js can only finish analyses for 84.4% and 84.0% of the packages; another two approaches ObjLupAnsys and ODGen can only analyze below 70% of the packages.

Cumulative Detected Vulnerabilities. Figure 8 shows the cumulative distribution function (CDF) plot of the percentage of packages where an approach finishes analysis (for the first 200 seconds): Forecast can finish the analysis for over 90% of the packages within 40 seconds, while no other approaches finish analysis for only 30% ~ 40% packages. For the first 10 seconds, Forecast completes the entire process for only a few packages, since it requires dynamically executing many JavaScript functions and identifying possible function call templates. The relatively longer execution time is therefore reasonable.

Figure 9 shows CDF plot of the percentage of packages for which an approach can detect the prototype pollution vulnerability. Within the first 40 seconds, Forecast can detect 92% of the packages (the analysis for 2% of the vulnerabilities has been finished, but cannot be detected since Forecast does not find the potential function call template). By contrast, none of the other four approaches can detect or generate working exploits for over 50% of the vulnerabilities within the first 40 seconds. These results illustrate Forecast 's better scalability.

Execution Time in Specific Packages. Table 5 presents the time consumption for 10 selected packages. Notably, ObjLupAnsys times out on 8 packages, ODGen times out on 9 packages, and Graph.js still times out on 3 packages. Worse still, even for very small packages with < 100 LoC, including *merge-objects* and *blindfold*, the approaches, except Forecast, may time out due to inefficient implementations of abstract interpretation and constraint solving.

[10]https://cveform.mitre.org

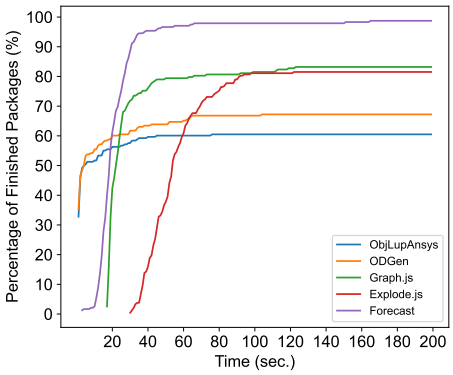


Fig. 8. CDF plot of the percentage of finished packages

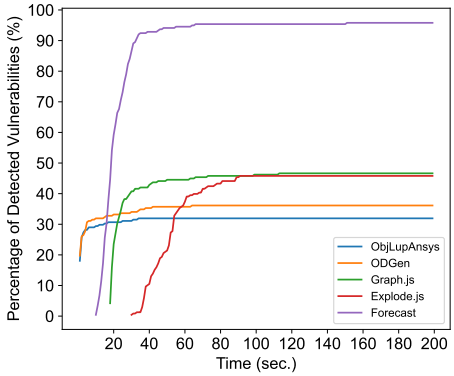


Fig. 9. CDF plot of the percentage of detected (exploited) vulnerabilities

Table 5. Execution time comparison between Forecast, ObjLupAnsys, ODGen, Graph.js and Explode.js on part of the JavaScript packages. The timeout is set to 5 minutes; LoC stands for Lines of Code.

Package	LoC	ObjLupAnsys	ODGen	Graph.js	Explode.js	Forecast
yargs-parser 6.0.0	799	timed out	timed out	timed out	8 sec.	32 sec.
lodash 4.17.11	17,108	timed out	timed out	42 sec.	timed out	127 sec.
think-helper 1.1.0	503	timed out	timed out	31 sec.	48 sec.	21 sec.
json-pointer 0.6.1	232	4 sec.	timed out	20 sec.	timed out	14 sec.
sds 3.2.0	324	timed out	timed out	29 sec.	54 sec.	18 sec.
fast-json-patch 2.0.4	969	timed out	timed out	226 sec.	53 sec.	14 sec.
simpl-schema 1.10.0	3,566	timed out	timed out	24 sec.	43 sec.	45 sec.
merge-objects 1.0.5	42	timed out	timed out	18 sec.	36 sec.	33 sec.
minimist 1.0.0	205	timed out	timed out	timed out	45 sec.	22 sec.
blindfold 1.0.1	51	1 sec.	1 sec.	timed out	timed out	11 sec.

Example 4.1. We here consider the package *blindfold* 1.0.1 as an example. We present part of the code in Figure 10. The sink location is at line 10: `base=base[path[i]]`. ObjLupAnsys and ODGen can detect this vulnerability within 1 second. However, Graph.js times out in the graph query stage. By looking into the code, the reason is that Graph.js (as well as Exlode.js) does not deal with the recursive calls in the right part in a proper way: it cannot stop from iteratively analyzing the function `blindfold`. Graph.js can deal with the package with the right part removed.

For the largest package *lodash* (with over 17 kLoC before compression), except Forecast, only Graph.js can finish the analysis within 5 minutes (however, it does not detect the vulnerability); Although Forecast spends more time dealing with the package, it is reasonable since *lodash* provides many functions. Forecast avoids the time-consuming abstraction for the whole program. Notice that since Forecast needs dynamic execution, the relatively longer execution time for some packages is reasonable. Nevertheless, in addition to effectiveness, Forecast still has better scalability than previous approaches.

Takeaway: Forecast achieves higher efficiency by successfully dealing with 98.7% packages within the time bound of 5 min.

```
function blindfold(base,path,value) {
  var ctx = base = base || this;

  path = path.split('.');
  for (var i=0; i<path.length; i++) {
    if (typeof base === 'undefined' ||
          base === null) {
      return base;
    }
    if (value && i===path.length-1) {
      base[path[i]] = value;
    }
    base = base[path[i]];
  }
  if (typeof base==='function') {
    path.pop();

    if (path.length) {
      base = base.bind(
      blindfold(ctx, path.join('.'))
      );
    } else {
      base = base.bind(ctx);
    }
  }
  return base;
};
```

Fig. 10. Part of the code of *blindfold* 1.0.1: the right part causes Graph.js time out.

5 DISCUSSIONS

In this section, we outline the current limitations of Forecast and the threats to validity.

5.1 Limitations

Design of Forced Execution. The major factor that affects the effectiveness of Forecast is forced execution, which opts for neither precision nor soundness. Like prior works [19, 21, 33], Forecast still produces infeasible execution paths. In the first stage, Forecast reports *potential* sink locations, and they may not actually be reachable. For example, if in some program location, an expression e can be evaluated to $\hat{\mathbf{s}}$, however, in actuality, it cannot be set to any string (including `"__proto__"`) that is tainted by the source, since it is preprocessed, e.g., via the predicate `e!="__proto__"` or `e.toUpperCase()==e`. Forecast may spend much time finding sinks via infeasible paths, which may yield a series of unrealizable function call templates, and further increase the overall time consumption. Forecast can be optimized by mitigating such infeasible paths in the first stage.

Design of Format Strings. In fact, format string is critical in synthesizing working exploit (without such mechanism, Forecast can only generate 146 working exploits for Benchmark 1). However, meta-templates only contribute to a small part of packages requiring special formats of input, e.g., *iniparser*. Forecast cannot deal with some real-world packages that require more complicated input format that must be expressed by a stronger grammar, however, it is beyond the assumption of this work.

Scalability on Large Packages. Forecast may incur significant analysis time on packages that export a large number of functions. For instance, *phpjs* provides hundreds of JavaScript implementations of PHP functions, requiring Forecast to analyze each function to determine its potential to propagate tainted strings. This exhaustive exploration can impact scalability on such codebases. Forecast can be further improved by selectively analyzing a subset of functions.

5.2 Threats to Validity

External Threats. A potential threat to external validity lies in the selection of benchmarks; the observed effectiveness of Forecast and competing techniques may not generalize to all JavaScript programs. However, the chosen benchmarks are representative of real-world JavaScript libraries known to be susceptible to prototype pollution, with substantial development histories

and numerous downstream dependents. We also use the latest versions of these packages, which better reflect the performance of Forecast in real-world programs.

Construct Threats. The potential threats to construct validity lie in (i) identification of vulnerability paths: the annotated information in the curated vulnerability set and our analysis of detected vulnerabilities may be incorrect, (ii) suitability of the selected metrics, e.g., an execution time measure may not be reasonable since it is not stable in different executions. To mitigate the threat, we carefully review the vulnerability information in Benchmark 1 and check that the approaches correctly identify the attack paths. Additionally, we re-evaluate the strategies for the selected packages to ensure consistent results across executions.

6 RELATED WORK

In this section, we review the literature on program analysis for securing JavaScript applications, as well as approaches to detecting and mitigating taint-style vulnerabilities.

6.1 Program Analysis for JavaScript Security

Taint-style vulnerabilities, including command injection, code injection, and path traversal, are well-known security issues in JavaScript community and have been extensively studied. Tools such as Graph.js [8], Explode.js [25], and ODGen [23] employ static or hybrid analyses to detect such vulnerabilities in JavaScript codebases.

Static and dynamic program analyses have been widely applied to detect security vulnerabilities in JavaScript applications. For instance, Nodest [31] builds on top of the abstract interpretation-based TAJS framework [12] to accelerate multi-module taint analysis. Li et al. leveraged Nodest to detect prototype pollution vulnerabilities, introducing the specialized tool PPNodest [22]. Madsen et al. [24] constructed event-based call graphs by extending JSAI [17] and TAJS, enabling the detection of event-related bugs. Jam [32], also based on TAJS, supports modular call graph construction, improving the scalability of vulnerability detection. Besides, rule-based lightweight linters such as ESLint [7] and JSHint [13] are widely used in practice to enforce coding conventions and detect common bugs; however, they are not designed for in-depth security analysis.

While static analysis provides soundness guarantees under certain assumptions, it often struggles with JavaScript's dynamic features, such as dynamic property accesses and runtime code generation. To address these limitations, dynamic analysis has been explored. Karim et al. [16] proposed a platform-independent dynamic taint analysis based on code instrumentation that can detect a range of taint-style vulnerabilities. The tool Taser proposed by Staicu et al. [39] also use dynamic analysis to support taint specification inference and detect potential security risks. However, unlike execution-based approaches, taint analysis lacks awareness of runtime behavior for arguments with unknown values. Inspired by these approaches, Forecast adopts a lightweight fuzzing strategy that reduces the effort required for manual language modeling and can effectively find exploits for the vulnerabilities.

The other series of works aims at mitigating taint-style vulnerabilities by enforcing security policies. Staicu et al. [38] conducted an empirical study of injection attacks in the Node.js ecosystem and proposed Synode, a mitigation framework that combines static analysis with runtime enforcement of security policies. Like Forecast, Synode employs a template-based mechanism; however, its goal is to synthesize runtime policies for safe module usage, rather than to generate exploits. Besides, GateKeeper, provided by Guarnieri et al. [10], an instrumentation framework provided by Yu et al. [41], and the approach provided by Jensen et al. [11] can enforce security policies by rewriting JavaScript code.

6.2 Prototype Pollution

Prototype pollution vulnerabilities, distinct from common taint-style vulnerabilities, have also received extensive attention in both server-side and client-side JavaScript applications. In addition to detection techniques [22, 23, 25], recent work has focused on identifying the gadgets that enable further exploitation of prototype pollution. Cornelissen et al. [5] developed a pipeline for mining such gadgets. Shcherbakov et al. introduced Dasty [36], which employs dynamic taint analysis to track gadget flows within the software supply chain. Kang et al. proposed GaLa [15], a system for detecting client-side prototype pollution gadgets in real-world websites. Further, Shcherbakov et al. [35] demonstrated how these gadgets can escalate to remote code execution, highlighting the severity of such vulnerabilities. However, the techniques of detecting gadgets are different from those for detecting prototype pollution, since gadgets can only take effect when a prototype object is actually polluted.

6.3 Forced Execution in Fuzzing and Dynamic Analysis

Besides Forecast, other analyses also leverage forced execution for different purposes. While forced executions in prior works can create infeasible paths, leading to imprecise results, they can also expand the code coverage and detect more potential issues.

Laursen et al. proposed an abstract interpretation framework, called approximate interpretation [21], that used forced execution to reduce unsoundness in the JavaScript call graph. In the forced execution phase, an unknown argument is represented by a special global object $\mathbf{p}^{\star}$, and any operation on the global object still returns this object. However, approximate interpretation is mainly used for call graph construction, which concerns primarily the deterministic facts in the JavaScript program [34], that is, the observed facts are the same every time the program is executed. Hence, the infeasible paths are not very critical in this case, as they typically resolve to a call site that always points to a specific function.

J-Force [19] and X-Force [33] also employ forced execution with little consideration for path feasibility. Their primary purpose is to detect additional vulnerabilities by recovering from runtime crashes that interrupt execution. Hence, they do not insist on precision and soundness, but on practicality. Note that, unlike Laursen et al., the two tools can explore execution paths to achieve higher code coverage, rather than executing the program's source code once and only collecting deterministic runtime hints.

7 CONCLUSION AND FUTURE WORK

We have introduced Forecast, a hybrid fuzzing approach to detect and generate exploits for prototype pollution vulnerabilities in JavaScript programs. Forecast utilizes forced execution to infer function call templates, and then refines such templates by generating a pool of template strings to replace the symbolic strings. By filling the concrete strings into the placeholders, Forecast can finally construct concrete exploits. Our evaluation using both benchmarks from prior works and real-world *npm* packages shows that Forecast achieves higher effectiveness and performance than prior techniques. In future work, we plan to extend Forecast to support more complex scenarios, e.g., prototype pollutions that lead to other severe vulnerabilities. We also plan to generalize it to support additional vulnerability classes and enhance the forced-execution engine.

DATA AVAILABILITY

The related data and code are available at [20].

ACKNOWLEDGMENTS

This work is supported by the Zhejiang Pioneer (Jianbing) Project (2025C01198), the National Science Foundation of China (No. 62372398, No. 72342025, No. 62302434), Zhejiang Provincial Science Foundation of China (No. LQK26F020002), and the Fundamental Research Funds for the Central Universities (No. 226-2025-00067).

REFERENCES

[1] Masudul Hasan Masud Bhuiyan, Adithya Srinivas Parthasarathy, Nikos Vasilakis, Michael Pradel, and Cristian-Alexandru Staicu. 2023. SecBench.js: An Executable Security Benchmark Suite for Server-Side JavaScript. In *Proceedings of the 45th International Conference on Software Engineering* (Melbourne, Victoria, Australia) *(ICSE '23)*. IEEE Press, 1059–1070. doi:10.1109/ICSE48619.2023.00096

[2] Tiago Brito, Mafalda Ferreira, Miguel Monteiro, Pedro Lopes, Miguel Barros, José Fragoso Santos, and Nuno Santos. 2023. Study of JavaScript Static Analysis Tools for Vulnerability Detection in Node.js Packages. *IEEE Transactions on Reliability* 72, 4 (2023), 1324–1339. doi:10.1109/TR.2023.3286301

[3] Darion Cassel, Wai Tuck Wong, and Limin Jia. 2023. NodeMedic: End-to-End Analysis of Node.js Vulnerabilities with Provenance Graphs. In *2023 IEEE 8th European Symposium on Security and Privacy (EuroS&P)*. 1101–1127. doi:10.1109/EuroSP57164.2023.00068

[4] Madhurima Chakraborty, Renzo Olivares, Manu Sridharan, and Behnaz Hassanshahi. 2022. Automatic Root Cause Quantification for Missing Edges in JavaScript Call Graphs. In *36th European Conference on Object-Oriented Programming (ECOOP 2022) (Leibniz International Proceedings in Informatics (LIPIcs), Vol. 222)*, Karim Ali and Jan Vitek (Eds.). Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl, Germany, 3:1–3:28. doi:10.4230/LIPIcs.ECOOP.2022.3

[5] Eric Cornelissen, Mikhail Shcherbakov, and Musard Balliu. 2024. GHunter: Universal Prototype Pollution Gadgets in JavaScript Runtimes. In *33rd USENIX Security Symposium (USENIX Security 24)*. USENIX Association, Philadelphia, PA, 3693–3710. https://www.usenix.org/conference/usenixsecurity24/presentation/cornelissen

[6] CVE. 2025. CVE database. https://www.cve.org/.

[7] ESLint. 2025. ESLint. https://eslint.org.

[8] Mafalda Ferreira, Miguel Monteiro, Tiago Brito, Miguel E. Coimbra, Nuno Santos, Limin Jia, and José Fragoso Santos. 2024. Efficient Static Vulnerability Analysis for JavaScript with Multiversion Dependency Graphs. *Proc. ACM Program. Lang.* 8, PLDI, Article 164 (June 2024), 25 pages. doi:10.1145/3656394

[9] GitHub. 2025. GitHub Advisories. https://github.com/advisories.

[10] Salvatore Guarnieri and V Benjamin Livshits. 2009. GATEKEEPER: Mostly Static Enforcement of Security and Reliability Policies for JavaScript Code. In *USENIX Security Symposium*, Vol. 10. 78–85.

[11] Simon Holm Jensen, Peter A. Jonsson, and Anders Møller. 2012. Remedying the eval that men do. In *Proceedings of the 2012 International Symposium on Software Testing and Analysis* (Minneapolis, MN, USA) *(ISSTA 2012)*. Association for Computing Machinery, New York, NY, USA, 34–44. doi:10.1145/2338965.2336758

[12] Simon Holm Jensen, Magnus Madsen, and Anders Møller. 2011. Modeling the HTML DOM and browser API in static analysis of JavaScript web applications. In *Proceedings of the 19th ACM SIGSOFT symposium and the 13th European conference on Foundations of software engineering*. 59–69.

[13] JSHint. 2024. JSHint. https://jshint.com.

[14] Mingqing Kang, Yichao Xu, Song Li, Rigel Gjomemo, Jianwei Hou, V. N. Venkatakrishnan, and Yinzhi Cao. 2023. Scaling JavaScript Abstract Interpretation to Detect and Exploit Node.js Taint-style Vulnerability. In *2023 IEEE Symposium on Security and Privacy (SP)*. 1059–1076. doi:10.1109/SP46215.2023.10179352

[15] Zifeng Kang, Muxi Lyu, Zhengyu Liu, Jianjia Yu, Runqi Fan, Song Li, and Yinzhi Cao. 2025. Follow My Flow: Unveiling Client-Side Prototype Pollution Gadgets from One Million Real-World Websites. In *2025 IEEE Symposium on Security and Privacy (SP)*. 991–1008. doi:10.1109/SP61157.2025.00016

[16] Rezwana Karim, Frank Tip, Alena Sochůrková, and Koushik Sen. 2020. Platform-Independent Dynamic Taint Analysis for JavaScript. *IEEE Transactions on Software Engineering* 46, 12 (2020), 1364–1379. doi:10.1109/TSE.2018.2878020

[17] Vineeth Kashyap, Kyle Dewey, Ethan A. Kuefner, John Wagner, Kevin Gibbons, John Sarracino, Ben Wiedermann, and Ben Hardekopf. 2014. JSAI: a static analysis platform for JavaScript. In *Proceedings of the 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering* (Hong Kong, China) *(FSE 2014)*. Association for Computing Machinery, New York, NY, USA, 121–132. doi:10.1145/2635868.2635904

[18] Hee Yeon Kim, Ji Hoon Kim, Ho Kyun Oh, Beom Jin Lee, Si Woo Mun, Jeong Hoon Shin, and Kyounggon Kim. 2022. DAPP: automatic detection and analysis of prototype pollution vulnerability in Node. js modules. *International Journal of Information Security* 21, 1 (2022), 1–23.

[19] Kyungtae Kim, I Luk Kim, Chung Hwan Kim, Yonghwi Kwon, Yunhui Zheng, Xiangyu Zhang, and Dongyan Xu. 2017. J-Force: Forced Execution on JavaScript. In *Proceedings of the 26th International Conference on World Wide Web* (Perth, Australia) *(WWW '17)*. International World Wide Web Conferences Steering Committee, Republic and Canton of Geneva, CHE, 897–906. doi:10.1145/3038912.3052674

[20] Dezhen Kong. 2026. Forecast Repository. https://github.com/cstimkong/forecast.

[21] Mathias Rud Laursen, Wenyuan Xu, and Anders Møller. 2024. Reducing Static Analysis Unsoundness with Approximate Interpretation. *Proc. ACM Program. Lang.* 8, PLDI, Article 194 (June 2024), 24 pages. doi:10.1145/3656424

[22] Song Li, Mingqing Kang, Jianwei Hou, and Yinzhi Cao. 2021. Detecting Node.js prototype pollution vulnerabilities via object lookup analysis. In *Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering* (Athens, Greece) *(ESEC/FSE 2021)*. Association for Computing Machinery, New York, NY, USA, 268–279. doi:10.1145/3468264.3468542

[23] Song Li, Mingqing Kang, Jianwei Hou, and Yinzhi Cao. 2022. Mining Node.js Vulnerabilities via Object Dependence Graph and Query. In *31st USENIX Security Symposium (USENIX Security 22)*. USENIX Association, Boston, MA, 143–160. https://www.usenix.org/conference/usenixsecurity22/presentation/li-song

[24] Magnus Madsen, Frank Tip, and Ondřej Lhoták. 2015. Static analysis of event-driven Node.js JavaScript applications. In *Proceedings of the 2015 ACM SIGPLAN International Conference on Object-Oriented Programming, Systems, Languages, and Applications* (Pittsburgh, PA, USA) *(OOPSLA 2015)*. Association for Computing Machinery, New York, NY, USA, 505–519. doi:10.1145/2814270.2814272

[25] Filipe Marques, Mafalda Ferreira, André Nascimento, Miguel E. Coimbra, Nuno Santos, Limin Jia, and José Fragoso Santos. 2025. Automated Exploit Generation for Node.js Packages. *Proc. ACM Program. Lang.* 9, PLDI, Article 201 (June 2025), 26 pages. doi:10.1145/3729304

[26] Mitre. 2025. CWE 1321. https://cwe.mitre.org/data/definitions/1321.html.

[27] Anders Møller, Benjamin Barslev Nielsen, and Martin Toldam Torp. 2020. Detecting locations in JavaScript programs affected by breaking library changes. *Proc. ACM Program. Lang.* 4, OOPSLA, Article 187 (Nov. 2020), 25 pages. doi:10.1145/3428255

[28] Anders Møller and Martin Toldam Torp. 2019. Model-based testing of breaking changes in Node.js libraries. In *Proceedings of the 2019 27th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering* (Tallinn, Estonia) *(ESEC/FSE 2019)*. Association for Computing Machinery, New York, NY, USA, 409–419. doi:10.1145/3338906.3338940

[29] Mozilla. 2025. EncodeURI. https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/encodeURI.

[30] Mozilla. 2025. Object.keys. https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Object/keys.

[31] Benjamin Barslev Nielsen, Behnaz Hassanshahi, and François Gauthier. 2019. Nodest: feedback-driven static analysis of Node.js applications. In *Proceedings of the 2019 27th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering* (Tallinn, Estonia) *(ESEC/FSE 2019)*. Association for Computing Machinery, New York, NY, USA, 455–465. doi:10.1145/3338906.3338933

[32] Benjamin Barslev Nielsen, Martin Toldam Torp, and Anders Møller. 2021. Modular call graph construction for security scanning of Node.js applications. In *Proceedings of the 30th ACM SIGSOFT International Symposium on Software Testing and Analysis* (Virtual, Denmark) *(ISSTA 2021)*. Association for Computing Machinery, New York, NY, USA, 29–41. doi:10.1145/3460319.3464836

[33] Fei Peng, Zhui Deng, Xiangyu Zhang, Dongyan Xu, Zhiqiang Lin, and Zhendong Su. 2014. X-Force: Force-Executing Binary Programs for Security Applications. In *Proceedings of the 23rd USENIX Security Symposium, San Diego, CA, USA, August 20-22, 2014*, Kevin Fu and Jaeyeon Jung (Eds.). USENIX Association, 829–844. https://www.usenix.org/conference/usenixsecurity14/technical-sessions/presentation/peng

[34] Max Schäfer, Manu Sridharan, Julian Dolby, and Frank Tip. 2013. Dynamic determinacy analysis. In *Proceedings of the 34th ACM SIGPLAN Conference on Programming Language Design and Implementation* (Seattle, Washington, USA) *(PLDI '13)*. Association for Computing Machinery, New York, NY, USA, 165–174. doi:10.1145/2491956.2462168

[35] Mikhail Shcherbakov, Musard Balliu, and Cristian-Alexandru Staicu. 2023. Silent Spring: Prototype Pollution Leads to Remote Code Execution in Node.js. In *32nd USENIX Security Symposium (USENIX Security 23)*. USENIX Association, Anaheim, CA, 5521–5538. https://www.usenix.org/conference/usenixsecurity23/presentation/shcherbakov

[36] Mikhail Shcherbakov, Paul Moosbrugger, and Musard Balliu. 2024. Unveiling the Invisible: Detection and Evaluation of Prototype Pollution Gadgets with Dynamic Taint Analysis. In *Proceedings of the ACM Web Conference 2024* (Singapore, Singapore) *(WWW '24)*. Association for Computing Machinery, New York, NY, USA, 1800–1811. doi:10.1145/3589334.3645579

[37] Snyk. 2025. Snyk.io. https://security.snyk.io.

[38] Cristian-Alexandru Staicu, Michael Pradel, and Benjamin Livshits. 2018. Synode: Understanding and Automatically Preventing Injection Attacks on Node.js. In *25th Annual Network and Distributed System Security Symposium, NDSS 2018, San Diego, California, USA, February 18-21, 2018*, Vol. 1. The Internet Society, 12–52.

[39] Cristian-Alexandru Staicu, Martin Toldam Torp, Max Schäfer, Anders Møller, and Michael Pradel. 2020. Extracting taint specifications for JavaScript libraries. In *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering* (Seoul, South Korea) *(ICSE '20)*. Association for Computing Machinery, New York, NY, USA, 198–209. doi:10.1145/3377811.3380390

[40] Webpack. 2025. Webpack. https://webpack.js.org.

[41] Dachuan Yu, Ajay Chander, Nayeem Islam, and Igor Serikov. 2007. JavaScript instrumentation for browser security. In *Proceedings of the 34th Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages* (Nice, France) *(POPL '07)*. Association for Computing Machinery, New York, NY, USA, 237–249. doi:10.1145/1190216.1190252